

Pontifícia Universidade Católica do Rio Grande do Sul

Escola Politécnica — Laboratório de Redes de Computadores

Protocolo de Transporte Confiável sobre UDP

Relatório Técnico — Trabalho Final

Diogo Pessin Camargo

Enzo Augusto Tonatto

Fernanda Farias Ubeti

22/06/2026

1 Visão Geral do Protocolo

O SRTP (*Simple Reliable Transport Protocol*) opera sobre UDP e implementa três mecanismos de controle de fluxo e erros: *Stop-and-Wait* (SAW), *Go-Back-N* (GBN) e *Selective Repeat* (SR). O cabeçalho de 9 bytes contém os campos SYN, FIN, SEQ (14 bits), ACK_flag, NACK, ACK (14 bits), Length (8 bits) e CRC32 (32 bits). Pacotes com CRC inválido são descartados silenciosamente; o *timeout* de 100 ms dispara a retransmissão. O handshake de três vias negocia a janela efetiva (mínimo entre as janelas propostas pelas duas partes).

2 Parte 1 — Stop-and-Wait

2.1 Throughput Teórico Máximo

No SAW, o emissor aguarda o ACK antes de enviar o próximo pacote. A eficiência do protocolo é dada por (Kurose & Ross, Seção 3.4):

$$\eta_{\text{SAW}} = \frac{L/R}{RTT + L/R} = \frac{1}{1 + 2a} \quad (1)$$

onde L é o tamanho do pacote em bits, R a taxa do enlace e $a = T_{\text{prop}}/T_{\text{trans}}$ é a razão entre o atraso de propagação e o tempo de transmissão.

Parâmetros da implementação:

- Payload máximo: 255 bytes; cabeçalho: 9 bytes $\Rightarrow L_{\text{pkt}} = 264 \text{ bytes} = 2112 \text{ bits}$
- Taxa do enlace R : Ethernet Gigabit ($R = 10^9 \text{ bps}$)
- Tempo de transmissão: $T_{\text{trans}} = 2112/10^9 \approx 2,1 \mu\text{s}$

RTT medido no cenário L0/P0: aproximadamente 0,116 ms, obtido a partir de pares DATA/ACK na captura `saw_L0.pcapng`. No par analisado, o pacote de dados foi observado em 19,358442500 s e o ACK correspondente em 19,358558500 s, resultando em RTT de 0,000116 s.

Com $RTT = 0,116 \text{ ms}$, tem-se $a = RTT/(2 T_{\text{trans}}) \approx 27,5$ e, pela Equação 1:

$$\eta_{\text{SAW}} = \frac{1}{1 + 2 \times 27,5} \approx 1,8\% \quad \Rightarrow \quad S_{\text{teórico}} = \eta \cdot R \approx 18000 \text{ kbps}$$

No cenário L0/P0, a transferência de 12800 bytes (51 pacotes) foi concluída com throughput medido de 3629,4 kbps e 0 retransmissões. A captura `saw_L0.pcapng` foi salva como evidência, e o hash MD5 do arquivo recebido coincidiu com o hash do arquivo original.

2.2 Cenários de Latência (L0–L3)

Latência adicional inserida com *Clumsy* no sender ou receiver. A Tabela 1 apresenta os resultados.

Tabela 1: SAW — Throughput e retransmissões sob latência artificial.

Cenário	Latência adicional	Throughput medido	Retransmissões
L0	0 ms (baseline)	3629,4 kbps	0
L1	50 ms	23,7 kbps	0
L2	100 ms (95 ms efetivo)	18,4 kbps	41
L3	150 ms	não concluiu	timeout contínuo

Nos cenários de latência, o throughput caiu de 3629,4 kbps em L0 para 23,7 kbps em L1, pois o SAW mantém apenas um pacote em trânsito e precisa aguardar um ACK antes de avançar. Em L2, a configuração exata de 100 ms no Clumsy coincidiu com o timeout fixo do protocolo e levou a retransmissões contínuas; por isso, foi usada uma aproximação prática de 95 ms

para obter uma medida final. Mesmo assim, o sender registrou 41 retransmissões e throughput de 18,4 kbps, evidenciando a transição crítica quando o atraso se aproxima do timeout. As capturas `saw_L1.pcapng` e `saw_L2.pcapng` mostram os ACKs chegando cada vez mais próximos do limite de 100 ms e a repetição de pacotes com o mesmo SEQ em L2. Em L3, com 150 ms de latência, a transferência não concluiu em tempo razoável: o atraso ficou maior que o timeout fixo, fazendo o sender retransmitir continuamente antes de receber confirmações úteis. A captura `saw_L3_timeout.pcapng` foi mantida como evidência desse comportamento de colapso do SAW quando o RTT efetivo excede o timeout.

2.3 Cenários de Perda (P0–P4)

Taxa de perda artificial inserida com *Clumsy*.

Tabela 2: SAW — Throughput e retransmissões sob perda de pacotes.

Cenário	Taxa de perda	Throughput medido	Retransmissões
P0	0% (baseline)	3629,4 kbps	0
P1	1%	846,9 kbps	1
P2	5%	188,5 kbps	5
P3	10%	150,8 kbps	5
P4	25%	72,0 kbps	12

O throughput esperado sob perda independente com probabilidade p é:

$$S_{\text{SAW}}(p) = \frac{(1-p)L}{RTT + L/R} \quad (2)$$

pois o número esperado de transmissões por pacote é $1/(1-p)$ (distribuição geométrica) e cada transmissão ocupa $RTT + L/R$ segundos.

Nos cenários de perda, o aumento da taxa de descarte reduziu o throughput e elevou as retransmissões. Em P1 (1%), houve apenas 1 retransmissão e o throughput medido foi 846,9 kbps. Em P2 (5%) e P3 (10%), foram observadas 5 retransmissões, com throughput de 188,5 kbps e 150,8 kbps, respectivamente. Em P4 (25%), o custo do SAW ficou mais evidente: 12 retransmissões e throughput de apenas 72,0 kbps. Como o protocolo envia um único pacote por vez, cada perda força o sender a aguardar o timeout de 100 ms antes de retransmitir. As capturas `saw_P1.pcapng` a `saw_P4.pcapng` mostram os intervalos sem ACK seguidos pela retransmissão do mesmo SEQ.

2.4 Comportamento do CRC32

O checksum CRC32 é calculado sobre o cabeçalho completo (com o campo CRC zerado) concatenado com o payload. Pacotes corrompidos são **descartados silenciosamente** pelo receiver sem envio de NACK, pois nenhum campo — inclusive o SEQ — é confiável. O *timeout* de 100 ms no sender dispara a retransmissão.

No teste com *Tamper* no Clumsy, a transferência SAW concluiu com throughput de 148,5 kbps e 5 retransmissões. A captura `saw_crc.pcapng` mostra pacotes de dados corrompidos que não receberam ACK imediato; após o timeout de 100 ms, o sender retransmitiu o mesmo SEQ. Esse comportamento confirma que o receiver descartou silenciosamente os pacotes cujo CRC32 recalculado não coincidia com o valor do cabeçalho. Sem o CRC32, esses payloads corrompidos poderiam ser aceitos como válidos, causando corrupção silenciosa no arquivo entregue à aplicação.

3 Parte 2 — Go-Back-N e Selective Repeat

3.1 Throughput Teórico sob Latência

Com janela de tamanho N , a eficiência do protocolo de janela deslizante é:

$$\eta_{\text{GBN/SR}} = \begin{cases} 1, & N \geq 1 + 2a \\ \frac{N}{1 + 2a}, & N < 1 + 2a \end{cases} \quad (3)$$

Para o cenário L0 ($RTT \approx 0,116$ ms), tem-se $1 + 2a \approx 56$. Com $N = 4$: $\eta \approx 7,1\%$. Com $N = 16$: $\eta \approx 28,6\%$.

3.2 Análise Comparativa sob Latência (SAW vs. GBN vs. SR)

Tabela 3: Throughput (kbps) dos três protocolos sob latência — janela $N = 4$.

Cenário	Latência	SAW	GBN (N=4)	SR (N=4)
L0	0 ms	3629,4	9830,6	10758,9
L1	50 ms	23,7	86,1	88,2
L2	100 ms	18,4	87,0 (70 ms)	não concluiu
L3	150 ms	não concluiu	não concluiu	não concluiu

Sob latência baixa, GBN e SR superaram o SAW por manterem múltiplos pacotes em trânsito: em L0, GBN atingiu 9830,6 kbps e SR 10758,9 kbps contra 3629,4 kbps do SAW. Em L1, a vantagem da janela também apareceu: GBN e SR ficaram em torno de 86–88 kbps, enquanto o SAW caiu para 23,7 kbps. Em latências próximas ou superiores ao timeout fixo de 100 ms, porém, todos os protocolos sofreram. O SAW apresentou retransmissões em L2 e não concluiu em L3; o GBN só concluiu com uma aproximação de 70 ms em L2; e o SR não concluiu em L2/L3. As capturas de timeout mostram que, quando os ACKs chegam depois do timeout, o sender retransmite antes de receber confirmações úteis.

3.3 Análise Comparativa sob Perda (GBN vs. SR)

Tabela 4: Throughput (kbps) e retransmissões sob perda — GBN e SR com $N = 4$ e $N = 16$.

Cen.	Perda	GBN			SR		
		N=4 kbps	N=4 retx	N=16 retx	N=4 kbps	N=4 retx	N=16 retx
P1	1%	855,0	0	137	842,4	0	2
P2	5%	13,3	100138	12705	820,3	14	34
P3	10%	não concluiu	timeout	16622	816,3	14	32
P4	25%	não concluiu	timeout	timeout	230,8	23	45

Sob perda, o GBN degradou muito mais rapidamente que o SR. Com $N = 4$, o GBN ficou em 855,0 kbps em P1, mas caiu para 13,3 kbps em P2, com 100138 retransmissões; em P3 e P4, não concluiu. Com $N = 16$, o GBN ainda concluiu P1, P2 e P3, mas com alto custo de retransmissão (137, 12705 e 16622 retransmissões), e não concluiu em P4. Isso ocorre porque, no GBN, a perda de um pacote obriga a retransmissão da janela a partir do pacote não confirmado. O SR foi mais estável: com $N = 4$, concluiu P1–P4 com throughput de 842,4, 820,3, 816,3 e 230,8 kbps; com $N = 16$, também concluiu todos os cenários, com retransmissões crescendo de 2 em P1 para 45 em P4. Os dados confirmam que a retransmissão seletiva reduz o custo de perda em comparação ao go-back em lote.

3.4 Análise Comparativa sob Reordenação (R0–R2)

Este é o ponto central da Parte 2. A reordenação de pacotes expõe a diferença fundamental de comportamento entre GBN e SR.

Tabela 5: Throughput (kbps) e retransmissões sob reordenação — $N = 4$ e $N = 16$.

Cen.	Reordenação	GBN		SR	
		N=4 (kbps / retx)	N=16 (kbps / retx)	N=4 (kbps / retx)	N=16 (kbps / retx)
R0	0%	14545,4 / 0	12854,9 / 0	6736,8 / 0	11112,2 / 0
R1	10%	não concluiu	timeout	não concluiu	timeout
R2	25%	não concluiu	timeout	não concluiu	timeout

GBN sob reordenação: o receiver descarta qualquer pacote recebido fora da ordem esperada e emite um NACK para o SEQ esperado. O sender, ao receber o NACK ou ao sofrer timeout, retransmite *toda a janela* a partir do SEQ não confirmado.

Nas capturas `gbn_R1_N4_timeout.pcapng`, `gbn_R2_N4_timeout.pcapng`, `gbn_R1_N16_timeout.pcapng` e `gbn_R2_N16_timeout.pcapng`, observa-se que a reordenação provocou recebimento fora da ordem esperada, descarte pelo receiver GBN e retransmissões em lote após NACK ou timeout. O baseline R0 concluiu bem para $N = 4$ e $N = 16$, mas

R1/R2 não concluíram em tempo razoável. O comportamento é coerente com o GBN: como o receiver não bufferiza fora de ordem, a reordenação é tratada como perda, e uma janela maior amplia o custo de cada evento.

SR sob reordenação: o receiver bufferiza o pacote fora de ordem dentro da janela, emite ACK individual para ele e aguarda o pacote faltante. Quando o pacote faltante chega, entrega ambos em ordem sem solicitar retransmissão.

Nas capturas `sr_R1_N4_timeout.pcapng`, `sr_R2_N4_timeout.pcapng`, `sr_R1_N16_timeout.pcapng` e `sr_R2_N16_timeout.pcapng`, os cenários com reordenação também não concluíram. Embora o SR implemente bufferização de pacotes fora de ordem, o padrão produzido pelo Clumsy gerou lacunas persistentes e timeouts suficientes para impedir a conclusão em tempo razoável. Assim, os resultados medidos mostram que a implementação conclui corretamente sem reordenação (R0), mas fica instável sob reordenação artificial intensa nesse ambiente de teste.

3.5 Impacto do Tamanho de Janela

O aumento de janela melhorou o potencial de throughput em cenários sem perda: em R0, GBN obteve 14545,4 kbps com $N = 4$ e 12854,9 kbps com $N = 16$, e SR obteve 6736,8 kbps com $N = 4$ e 11112,2 kbps com $N = 16$. Sob perda, porém, o efeito depende do protocolo. No GBN, a janela maior aumentou o custo dos eventos de perda: em P1, $N = 16$ já registrou 137 retransmissões; em P2/P3, foram 12705 e 16622; e em P4 houve timeout. No SR, a janela maior preservou melhor o throughput, mas aumentou o número absoluto de retransmissões em alguns cenários (por exemplo, P4 passou de 23 retransmissões em $N = 4$ para 45 em $N = 16$). Sob reordenação, os cenários R1/R2 não concluíram para ambos, indicando que a combinação entre reordenação artificial e timeout fixo foi severa para esta implementação.

3.6 Conclusão Comparativa

A Tabela 6 sintetiza as condições de rede em que cada protocolo é mais adequado, fundamentada nos dados coletados.

Tabela 6: Síntese comparativa dos três protocolos.

Condição de rede	SAW	GBN	SR
Baixa latência, sem perda	Adequado	Melhor	Melhor
Alta latência (> 50 ms)	Inadequado	Bom	Bom
Alta perda ($> 5\%$)	Ruim	Ruim (janela grande)	Bom
Alta reordenação	N/A	Ruim	Bom
Complexidade de implementação	Baixa	Média	Alta

Os resultados mostram que o SAW é simples e correto, mas pouco eficiente quando o RTT cresce: saiu de 3629,4 kbps em L0 para 23,7 kbps em L1, e não concluiu em L3. O GBN alcançou bom desempenho em cenários sem perda ou com perda baixa, mas degradou rapidamente quando houve perda moderada, devido à retransmissão em lote da janela. O SR apresentou o melhor comportamento sob perda, concluindo P1–P4 tanto com $N = 4$ quanto com $N = 16$, embora com aumento de retransmissões. Sob reordenação artificial, os cenários R1/R2 não concluíram em tempo razoável para GBN e SR, o que evidencia a sensibilidade da implementação ao padrão de reordenação gerado pelo Clumsy e ao timeout fixo de 100 ms.

4 Uso de Inteligência Artificial

Foram utilizadas ferramentas de inteligência artificial generativa durante o desenvolvimento do trabalho, principalmente ChatGPT/Codex e Claude. As ferramentas auxiliaram na interpretação do enunciado, revisão da conformidade do protocolo, organização dos experimentos, depuração de problemas de execução, estruturação do relatório e redação das análises.

Exemplos representativos de consultas incluem: como interpretar o formato de cabeçalho SRTP; como executar os testes com Clumsy e Wireshark; como analisar os efeitos de timeout de 100 ms nos cenários de latência; e como comparar SAW, GBN e SR sob perda e reordenação. As decisões finais, execução dos experimentos, coleta de capturas e validação por hash foram realizadas pelos integrantes do grupo. Também foram usadas como referência a Seção 3.4 de Kurose & Ross e a documentação técnica das ferramentas Python, Wireshark e Clumsy.